Django GET Parameters — Complete Beginner to Advanced Notes

1. What Is a GET Parameter?

A GET parameter is data sent through the URL query string. Django uses `request.GET` to access these values.

Example URL:

```
http://localhost:8000/products/?category=electronics&sort=price
```

Here:

- `category=electronics`
- `sort=price`

Django retrieves these values using `request.GET`.

```
category = request.GET.get("category")
sort_by = request.GET.get("sort")
```

Results:

```
category  # "electronics"
sort_by   # "price"
```

---

2. `request.GET.get()`

Use `request.GET.get()` to retrieve a single GET parameter.

```
category = request.GET.get("category")
```

If the URL is:

```
/products/?category=electronics
```

The result is:

```
category == "electronics"
```

If the parameter is missing:

```
/products/
```

Then:

```
category is None
```

You can check whether a value exists:

```
if category:
    print("Category:", category)
```

Important: GET parameters are strings by default. Validate and convert them when you need integers, decimals, dates, or other types.

---

3. Default Values

You can provide a default value to `get()`.

```
category = request.GET.get("category", "all")
```

Examples:

```
/products/
```

Result:

```
category == "all"
```

```
/products/?category=electronics
```

Result:

```
category == "electronics"
```

Note that the default applies when the key is missing, not when it is present with an empty value.

---

4. Multiple GET Parameters

You can retrieve several parameters from one URL.

```
category = request.GET.get("category")
sort_by = request.GET.get("sort")
search = request.GET.get("search")
```

URL:

```
/products/?category=electronics&sort=price&search=phone
```

Results:

```
category  # "electronics"
sort_by   # "price"
```

```
search    # "phone"
```

GET URL structure

```
/path/?key=value&key2=value2
```

- `?` begins the query string.
- `=` separates a key and its value.
- `&` separates multiple parameters.

---

5. What Is `request.GET`?

`request.GET` is a Django `QueryDict` object containing query-string parameters.

```
def product_list(request):
    print(request.GET)
```

For this URL:

```
/products/?category=electronics&sort=price
```

It contains values conceptually equivalent to:

```
{
    "category": ["electronics"],
    "sort": ["price"],
}
```

A `QueryDict` can hold multiple values for the same key.

```
/products/?tag=python&tag=django
```

Get the last value:

```
tag = request.GET.get("tag")
```

Get all values:

```
tags = request.GET.getlist("tag")
# ["python", "django"]
```

---

6. GET Parameters vs URL Path Parameters

These are two different ways of passing data to a Django view.

URL path parameter

URL:

```
/products/5/
```

URL configuration:

```
path("products/<int:id>/", views.product_detail, name="product_detail")
```

View:

```
def product_detail(request, id):
    ...
```

Django passes the path value through the view argument `id`.

GET parameter

URL:

```
/products/?category_id=5
```

View:

```
def product_list(request):
    category_id = request.GET.get("category_id")
```

Remember:

- Path parameters are passed to the view through URL arguments.
- GET parameters are accessed using `request.GET`.

---

7. Basic Product List Example

`views.py`

```
from django.shortcuts import render
from .models import Product


def product_list(request):
    category = request.GET.get("category")
    sort_by = request.GET.get("sort")

    products = Product.objects.all()

    if category:
        products = products.filter(category__name=category)

    if sort_by == "price":
        products = products.order_by("price")
```

```
    elif sort_by == "-price":
        products = products.order_by("-price")

    return render(
        request,
        "product_list.html",
        {"products": products},
    )
```

This example assumes `Product.category` is a ForeignKey to a `Category` model with a `name` field.

`urls.py`

```
from django.urls import path
from . import views

urlpatterns = [
    path("products/", views.product_list, name="product_list"),
]
```

The query parameters do not need to be declared in `urlpatterns`.
These URLs use the same route:

```
/products/
/products/?category=electronics
/products/?category=electronics&sort=price
```

`product_list.html`

```
<h1>Product List</h1>

{% for product in products %}
    <p>
        {{ product.name }} -
        {{ product.category.name }} -
        {{ product.price }}
    </p>
{% empty %}
    <p>No products found.</p>
{% endfor %}
```

---

8. GET Parameter Search

A search parameter allows users to search for products by name.

`views.py`

```
from django.shortcuts import render
from .models import Product


def product_list(request):
    search = request.GET.get("search", "").strip()

    products = Product.objects.all()

    if search:
        products = products.filter(name__icontains=search)

    return render(
        request,
        "product_list.html",
        {
            "products": products,
            "search": search,
        },
    )
```

URL:

```
/products/?search=phone
```

The `icontains` lookup searches for a case-insensitive partial match.

Template search form

```
<form method="get" action="{% url 'product_list' %}">
    <input
        type="text"
        name="search"
        value="{{ search }}"
        placeholder="Search products..."
    >
    <button type="submit">Search</button>
</form>
```

The browser sends the search term through the URL.

Example:

```
/products/?search=phone
```

---

9. Category Filtering

If `Product` has a regular text field for category, you can filter directly.

```
category = request.GET.get("category")

products = Product.objects.all()

if category:
    products = products.filter(category=category)
```

If `category` is a ForeignKey to a `Category` model, use a relationship lookup or the foreign-key ID.

```
products = products.filter(category__name=category)
```

Or:

```
category_id = request.GET.get("category_id")

if category_id:
    products = products.filter(category_id=category_id)
```

The second example needs validation if the ID is user-supplied.

---

10. Complete ForeignKey Example

`models.py`

```
from django.db import models
from django.utils.text import slugify


class Category(models.Model):
    name = models.CharField(max_length=255, unique=True)
    slug = models.SlugField(max_length=255, unique=True, blank=True)

    def save(self, *args, **kwargs):
        if not self.slug:
            self.slug = slugify(self.name)
        super().save(*args, **kwargs)

    def __str__(self):
```

```
        return self.name


class Project(models.Model):
    category = models.ForeignKey(
        Category,
        on_delete=models.CASCADE,
        related_name="projects",
    )
    name = models.CharField(max_length=255)
    description = models.TextField(blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.name
```

Note: If two category names produce the same slug, the unique constraint can cause an error. For production projects, generate unique slugs and handle collisions.

`views.py`

```
from django.shortcuts import render
from .models import Project


def project_list(request):
    category_id = request.GET.get("category_id")

    projects = Project.objects.select_related("category").all()

    if category_id:
        projects = projects.filter(category_id=category_id)

    return render(
        request,
        "projects.html",
        {"projects": projects},
    )
```

For this basic example, an invalid category ID string can raise a database conversion error. The robust version below validates the input.

Safer category ID validation

```
from django.shortcuts import render
from .models import Category, Project


def project_list(request):
    category_id = request.GET.get("category_id")
    search = request.GET.get("search", "").strip()

    projects = Project.objects.select_related("category").all()

    if category_id:
        if not category_id.isdigit():
            projects = projects.none()
        elif Category.objects.filter(pk=int(category_id)).exists():
            projects = projects.filter(category_id=int(category_id))
        else:
            projects = projects.none()

    if search:
        projects = projects.filter(name__icontains=search)

    return render(
        request,
        "projects.html",
        {
            "projects": projects,
            "search": search,
            "category_id": category_id,
        },
    )
```

This version treats invalid or nonexistent category IDs as an empty result set. You could instead return a validation message or HTTP 400 response, depending on the application's requirements.

`urls.py`

```
from django.urls import path
from . import views

urlpatterns = [
    path("projects/", views.project_list, name="project_list"),
]
```

`projects.html`

```
<h1>Projects</h1>

<form method="get" action="{% url 'project_list' %}">
    <input
        type="text"
        name="search"
        value="{{ search }}"
        placeholder="Search projects..."
    >

    <input
        type="number"
        name="category_id"
        value="{{ category_id }}"
        min="1"
        placeholder="Category ID"
    >

    <button type="submit">Filter</button>
</form>

{% for project in projects %}
    <article>
        <h2>{{ project.name }}</h2>
        <p>Category: {{ project.category.name }}</p>
        <p>{{ project.description }}</p>
    </article>
{% empty %}
    <p>No projects found.</p>
{% endfor %}
```

Example URLs:

```
/projects/
/projects/?category_id=1
/projects/?category_id=2
/projects/?search=django
/projects/?category_id=1&search=django
```

`select_related("category")` fetches the related category in the same SQL query, avoiding an extra database query for each project's category.

---

11. Dynamic Category Filter Links

Instead of manually writing category IDs, generate links from category objects.

View

```
from django.shortcuts import render
from .models import Category, Project


def project_list(request):
    category_id = request.GET.get("category_id")

    projects = Project.objects.select_related("category").all()
    categories = Category.objects.all()

    if category_id and category_id.isdigit():
        projects = projects.filter(category_id=int(category_id))

    return render(
        request,
        "projects.html",
        {
            "projects": projects,
            "categories": categories,
            "selected_category_id": category_id,
        },
    )
```

Template

```
<h2>Categories</h2>

<a href="{% url 'project_list' %}">All Projects</a>

{% for category in categories %}
    <a href="{% url 'project_list' %}?category_id={{ category.id }}">
        {{ category.name }}
    </a>
{% endfor %}
```

Example generated link:

```
/projects/?category_id=3
```

For complex links containing multiple parameters, consider using Django's URL reversing and a query-string helper or preserve existing query parameters in the view/template logic.

---

12. Combining Search, Category Filter, and Sorting

A practical project list view can combine all three features.

```
from django.shortcuts import render
from .models import Category, Project


def project_list(request):
    category_id = request.GET.get("category_id", "")
    search = request.GET.get("search", "").strip()
    sort = request.GET.get("sort", "")

    projects = Project.objects.select_related("category").all()

    if category_id:
        if category_id.isdigit():
            projects = projects.filter(category_id=int(category_id))
        else:
            projects = projects.none()

    if search:
        projects = projects.filter(name__icontains=search)

    allowed_sorting = {
        "name": "name",
        "-name": "-name",
        "created": "created_at",
        "-created": "-created_at",
    }

    sort_field = allowed_sorting.get(sort)

    if sort_field:
        projects = projects.order_by(sort_field)

    categories = Category.objects.all()

    return render(
```

```
        request,
        "projects.html",
        {
            "projects": projects,
            "categories": categories,
            "search": search,
            "category_id": category_id,
            "sort": sort,
        },
    )
```

Example URL:

```
/projects/?category_id=1&search=django&sort=-created
```

The view applies the category filter, searches project names, and then sorts by newest first.

---

13. GET Sorting

For a product list, users may want to sort by price.

```
from django.shortcuts import render
from .models import Product


def product_list(request):
    sort = request.GET.get("sort", "")

    products = Product.objects.all()

    allowed_sorting = {
        "price": "price",
        "-price": "-price",
        "name": "name",
        "-name": "-name",
    }

    sort_field = allowed_sorting.get(sort)

    if sort_field:
        products = products.order_by(sort_field)

    return render(
```

```
        request,
        "product_list.html",
        {
            "products": products,
            "sort": sort,
        },
    )
```

URLs:

```
/products/?sort=price
/products/?sort=-price
/products/?sort=name
/products/?sort=-name
```

- `price` : ascending price.
- `-price` : descending price.
- `name` : ascending name.
- `-name` : descending name.

Security note: Never pass arbitrary user input directly into `order_by()` when you intend to allow only specific fields. Use an allowlist of permitted sorting options.

---

14. Product Detail Page Using an Integer ID

GET parameters are commonly used for filters and searches. A product's detail page often uses a path parameter.

`urls.py`

```
from django.urls import path
from . import views

urlpatterns = [
    path("products/", views.product_list, name="product_list"),
    path(
        "product/<int:id>/",
        views.product_detail,
        name="product_detail",
    ),
]
```

`views.py`

```
from django.shortcuts import render, get_object_or_404
from .models import Product


def product_detail(request, id):
    product = get_object_or_404(Product, id=id)

    return render(
        request,
        "product_detail.html",
        {"product": product},
    )
```

`get_object_or_404()` returns the object if it exists; otherwise, Django raises an HTTP 404 response instead of returning an unhandled `DoesNotExist` exception.

Link in `product_list.html`

```
{% for product in products %}
    <p>
        <a href="{% url 'product_detail' product.id %}">
            {{ product.name }}
        </a>
        - {{ product.price }}
    </p>
{% empty %}
    <p>No products found.</p>
{% endfor %}
```

`product_detail.html`

```
<h1>{{ product.name }}</h1>

<p>Category: {{ product.category }}</p>
<p>Price: {{ product.price }}</p>
<p>{{ product.description }}</p>

<a href="{% url 'product_list' %}">Back to Products</a>
```

Example URL:

```
/product/5/
```

Django passes `id=5` to the view.

---

15. Using Slugs Instead of IDs

A slug is a URL-friendly text identifier, such as `iphone-15`.

Model

```
from django.db import models


class Product(models.Model):
    name = models.CharField(max_length=255)
    slug = models.SlugField(unique=True)
    price = models.DecimalField(max_digits=10, decimal_places=2)

    def __str__(self):
        return self.name
```

Example URL:

```
/product/iphone-15/
```

`urls.py`

```
path(
    "product/<slug:slug>/",
    views.product_detail,
    name="product_detail",
)
```

`views.py`

```
from django.shortcuts import render, get_object_or_404
from .models import Product


def product_detail(request, slug):
    product = get_object_or_404(Product, slug=slug)

    return render(
        request,
        "product_detail.html",
        {"product": product},
    )
```

Link in the template

```
<a href="{% url 'product_detail' product.slug %}">
    {{ product.name }}
</a>
```

A slug must be populated and unique for this example to work.

ID vs Slug

ID	Slug
`/product/5/`	`/product/iphone-15/`
Short and simple	Human-readable
Uses a database identifier	Uses a meaningful string
Common for internal resources	Common for content pages

Both are valid approaches. A slug is not automatically better for SEO; useful content, stable URLs, and the overall site structure also matter.

---

16. GET Forms

The `method="get"` attribute sends form values through the URL query string.

```
<form method="get" action="{% url 'product_list' %}">
    <input type="text" name="search" placeholder="Search products">
    <button type="submit">Search</button>
</form>
```

If a user searches for `laptop`, the browser may navigate to:

```
/products/?search=laptop
```

Use GET for read-only actions such as searching, filtering, sorting, and pagination.
Do not use GET for operations that modify data, such as deleting a product or updating an account. Use an appropriate method such as POST and apply Django's CSRF protection.

---

17. GET Parameters and Pagination

GET parameters can also represent a page number.

```
from django.core.paginator import Paginator
from django.shortcuts import render
from .models import Product


def product_list(request):
    products = Product.objects.all().order_by("id")

    paginator = Paginator(products, 10)
    page_number = request.GET.get("page")
    page_obj = paginator.get_page(page_number)

    return render(
        request,
        "product_list.html",
        {"page_obj": page_obj},
    )
```

Template:

```
{% for product in page_obj %}
    <p>{{ product.name }}</p>
{% empty %}
    <p>No products found.</p>
{% endfor %}

{% if page_obj.has_previous %}
    <a href="?page={{ page_obj.previous_page_number }}">Previous</a>
{% endif %}

<span>Page {{ page_obj.number }} of {{ page_obj.paginator.num_pages }}</span>

{% if page_obj.has_next %}
```

```
    <a href="?page={{ page_obj.next_page_number }}">Next</a>
{% endif %}
```

Example:

```
/products/?page=2
```

When combining pagination with search or filters, preserve those existing query parameters in the pagination links so the selected filters remain active.

---

18. Common Mistakes

Mistake 1: Using `request.GET` as a view argument

Incorrect:

```
def product_list(request, category_id):
    ...
```

This only works if the URL pattern includes a matching path parameter.
Correct for a GET query parameter:

```
def product_list(request):
    category_id = request.GET.get("category_id")
```

Mistake 2: Forgetting the `?`

Correct:

```
/products/?category_id=1
```

Incorrect:

```
/products/category_id=1
```

Mistake 3: Treating GET values as integers automatically

```
category_id = request.GET.get("category_id")
```

The value is a string, such as `"1"`. Validate it before converting it to an integer.

```
if category_id and category_id.isdigit():
    category_id = int(category_id)
```

For more complex validation, use a Django Form or another explicit validation layer.

Mistake 4: Using `Product.objects.get()` for a list

For a list of products:

```
products = Product.objects.filter(category_id=category_id)
```

For one product:

```
product = get_object_or_404(Product, id=id)
```

Mistake 5: Forgetting the template context

Incorrect:

```
return render(request, "projects.html")
```

Correct:

```
return render(
    request,
    "projects.html",
    {"projects": projects},
)
```

Mistake 6: Passing arbitrary sort fields

Avoid:

```
products = products.order_by(request.GET.get("sort"))
```

Prefer a whitelist of allowed fields, as shown in the sorting example.

---

19. Interview Questions and Answers

Q1. What is a GET parameter in Django?

It is a value passed through the URL query string and accessed with `request.GET`.

Q2. What is `request.GET.get("name")`?

It retrieves the value associated with the `name` query parameter, or `None` if the key is missing and no default is provided.

Q3. What is the difference between `request.GET` and `request.POST`?

`request.GET` accesses query-string parameters. `request.POST` accesses form-encoded POST data, such as data submitted through a typical HTML POST form.

Q4. Are GET parameter values integers?

No. They are generally strings and should be validated and converted when needed.

Q5. What does `request.GET.get("category", "all")` do?

It returns the value of `category`, using `"all"` if the key is absent.

Q6. What is a `QueryDict`?

It is Django's dictionary-like structure for request query parameters and form data. It supports multiple values for a single key.

Q7. How do you get all values of a repeated query parameter?

```
tags = request.GET.getlist("tag")
```

Q8. What is the difference between a URL path parameter and a GET parameter?

A path parameter is captured by the URL pattern and passed to the view as an argument. A GET parameter is retrieved from `request.GET`.

Q9. Why use `get_object_or_404()`?

It returns the requested object or raises an HTTP 404 response if no matching object exists.

Q10. What is `select_related()`?

It lets Django retrieve related ForeignKey or OneToOne objects in the same SQL query, reducing extra database queries.

Q11. Is GET suitable for deleting a record?

No. GET should be safe and read-only. Use POST or another appropriate method for state-changing actions, with the required protections.

Q12. Can a GET URL contain multiple parameters?

Yes.

```
/projects/?category_id=1&search=django&sort=-created
```

---

20. Quick Revision

Requirement	Django code
Get one parameter	`request.GET.get("search")`
Get a parameter with a default	`request.GET.get("page", "1")`

Get repeated values	`request.GET.getlist("tag")`
Filter a ForeignKey by ID	`Project.objects.filter(category_id=category_id)`
Search case-insensitively	`Project.objects.filter(name__icontains=search)`
Sort ascending	`Product.objects.order_by("price")`
Sort descending	`Product.objects.order_by("-price")`
Capture an integer path parameter	`path("product/<int:id>/", ...)`
Capture a slug path parameter	`path("product/<slug:slug>/", ...)`

Build a named URL	`{% url 'product_detail' product.id %}`
Safely retrieve one object	`get_object_or_404(Product, id=id)`
Optimize ForeignKey access	`Project.objects.select_related("category")`

Final Memory Map

```
GET Query Parameter
    request.GET.get("category_id")

URL Path Parameter
    /product/5/
    def product_detail(request, id):

Dynamic URL
    {% url 'product_detail' product.id %}

Search
    name__icontains=search

ForeignKey Filter
    category_id=category_id

Sorting
    order_by("price")
    order_by("-price")

Safe Object Retrieval
    get_object_or_404(Product, id=id)
```

```
Pagination
    request.GET.get("page")
```

The key idea: Use `request.GET` for search, filtering, sorting, and pagination; use URL path parameters for resources such as product or project detail pages. Validate user-supplied values and use an allowlist for user-selectable sorting fields.